Records Analytics API

FastAPI portfolio project | Practical setup and walkthrough

Manage records and produce category counts, status counts, and total amounts for dashboards.

1. What you will learn

Build REST endpoints, validate JSON requests with Pydantic, persist data in SQLite, return dashboard metrics, and test API behavior with isolated databases.

Included files

app/main.py - application and schemas
tests/test_api.py - API tests
sample.json - demo request
Dockerfile - container setup
README.md - commands and GitHub upload steps

Architecture

Swagger UI or API client sends JSON to FastAPI. Pydantic validates the request. Endpoint functions read or write SQLite using parameterized SQL. Responses return JSON. Database initialization runs in the application lifespan.

Scope

This is a local learning API. It includes interactive API documentation but no standalone frontend, login or user accounts. Use fictional data while learning.

2. Run the project

Install Python 3.10 or newer. Extract the ZIP and open a terminal inside the project folder. Choose the activation command for your operating system.

```
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# macOS/Linux:
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload
# In another activated terminal:
python -m pytest -q
```

Open http://127.0.0.1:8000/docs in your browser. Keep the server terminal running. If port 8000 is busy, add --port 8001 to the uvicorn command.

Docker alternative

```
docker build -t records-analytics-api .
docker run --rm -p 8000:8000 \
  -v records-analytics-api-data:/app/data records-analytics-api
```

The named volume preserves database contents when the container is removed. DATABASE_PATH can override the default data/app.db location.

3. Try a complete demo

In /docs, expand POST /records, select Try it out, paste this JSON and press Execute. Expect HTTP 201 and an id in the response.

```
{
  "title": "Property record review",
  "category": "deeds",
  "status": "pending",
  "amount": 2500
}
```

Next, run GET /records. Run GET /records/summary to inspect counts. Use GET /records/{item_id} with the returned id. Use PUT with all fields to change status. Use DELETE with that id to remove the item; a later GET returns 404.

Filtering and pagination

Example: /records?status=pending&limit=10&offset=0. Limit is capped at 100. Offset skips records. Results are ordered by newest id first. Summary metrics cover the entire dataset, not just the current page.

Errors to understand

422: missing fields, invalid status, unexpected fields, or invalid pagination. 404: record does not exist. 204: successful deletion with no response body. Empty summaries return total 0.

4. Publish and explain it

Create a new empty GitHub repository named records-analytics-api. Extract the ZIP, then upload the files inside the project folder using Add file / Upload files. README.md should appear at the repository root. Alternatively use the Git commands in README.md to preserve hidden files.

Verification

Run python -m pytest -q. Tests cover create, read, replace, delete, filtering, pagination, summary counts, invalid requests, missing records and persistence. Each test uses a temporary database rather than your local demo data.

Interview explanation

Explain the lifecycle initialization, schema validation, parameterized SQL, 201/204/404/422 responses, and the reason for synchronous SQLite endpoints. Explain why the project needs authentication, migrations and a stronger database before becoming a shared service.

Suggested next steps

Add a frontend, API authentication, per-user ownership, typed SQL tables, migrations, PostgreSQL and CI. Add date fields and trend reports. Customize one feature and add tests for it before sharing as your own portfolio work.

Sources

Official FastAPI request bodies: https://fastapi.tiangolo.com/tutorial/body/
Official FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/
Python SQLite: https://docs.python.org/3/library/sqlite3.html